FLAG LAB

Dissertation

Code as Art, Art as Code

The SCuLPT Programing Language

Nicolás Londoño Cuellar

May 28, 2026

This thesis is submitted in partial fulfilment of the requirements for a degree of Undergraduate in Systems and Computing Engineering.

Thesis Committee:

Prof. Nicolás Cardozo (Promotor) Universidad de los Andes, Colombia

Code as Art, Art as Code

The SCuLPT Programing Language

Systems and Computing Engineering Department

FLAG lab

Faculty of Engineering

Universidad de los Andes

Bogotá, Colombia

This manuscript has been set with the help of TeXShop and PDFLaTeX (with BibTeX support). Bugs have been tracked down in the text and squashed thanks to *Bugs in Writing* by Dupré [5] and *Elements of Style* by Strunk and White [18].

Thanks to everyone who made this possible and believed in the power of human computation.

Abstract

Everyone should have the opportunity to interact with code just as the average programmer does, no matter the condition or background they may have. The Simple Cubic Language for Programming Tasks (SCuLPT) is a programming language and artistic ecosystem designed to challenge the status quo of programming languages and computational thinking. SCuLPT disrupts the standard typewriter as the default interface for coding and programming languages with an intuitive, 3D interface and its specification. This is done by creating programming and artistic constructs that become tangible objects such that, when composed, generate computable programs and art pieces. Along the language, we provide a set of tools aimed to make art a tool for coding and programming an art form. Also included in the ecosystem exists the Simple Cubic Language for Programming Tasks Environment and Runtime (SCuLPTER) to run and test programs before assembling the final sculpture. The constructed language is a fully capable, Turing complete language that serves as a creative outlet as well. Using said tools, we perform an empirical validation of the experience with novices and expert programmers alike. The results show programming proficiency in solving small computational programs across several populations, and some really neat art!

Acknowledgements

Having so many to thank in such a small space is daunting, I do not want to skip anyone, nonetheless, I'll try to be succinct.

First and foremost, I would like to thank my advisor, Nicolas Cardozo, for listening to a weird idea from an even weirder student. His guidance has been incredible, not only in this work, but also in what follows from it.

I also thank the members of the FLAGLab. They gave me a space to talk about printed plastic pieces and esoteric programming languages. Your feedback have been invaluable in shaping this work and the language's technicalities. Making SCuLPT out of 3D printed pieces was a monumental task my poor Ender printer was simply incapable of, and I would like to thank the people at CREA and the guys running the department's 3D printers for their help in making this possible.

I will probably never be as good as a designer as my sister, Sara, but I will always be grateful for her help in making the SCuLPT visual language look amazing and work as well as they did. I also owe a lot to my mother, who has always supported me in my endeavours, even when they seem a bit out there.

I am not very good at organizing in my schedules. I would like to thank my partner, who has been there to mindlessly remind me of deadlines and to keep me on track by simply asking for updates. Your selfless interest on the project has been a great help in keeping me focused and on track. I am not sure how you put up with me, but I am glad you do.

Finally would like to thank my friends for putting up with me and my shenanigans, and for always being there to help me out when I needed it. Always at the ready to make me crack a smile or two when times got rough and my hands numb from hand-sanding every printed piece of SCuLPT.

Contents

List of Figures

Chapter 1

Introduction

The typewriter paradigm, through the keyboard and screen, has been the reigning interface to materialize programs into code since the beginning of programming as we know it today.

Its interface design lack what is needed to support every user. Naturally, users are not created equally. Prior work in the user experience and human-interface integration show reports in which technology and its designs can discriminate against users due to several factors, such as different physical or cognitive capacities, race and gender [10]. Moreover, programming has a steep learning curve and a high entry level [3]. These factors make programming languages and their interfaces inaccessible to a large portion of the population, disincentivizing them from learning to program and interact with code in the first place. Different approaches to computing are needed to lower the entry threshold and make computing more approachable to everyone.

Current programming languages are based on the same principles that we have used since the 50s. The same principles that were designed to be used with a typewriter and a flat surface in which to write. To lower the entry threshold of programming and computing, more approachable interfaces and methods are required. New fields in computing are emerging with different intents but still the same interfaces [22]. In order to explore new methods of computing, we must look further ahead than the tools we currently have and revisit how we approach code in general.

To do so, we must shift attention to other mediums that can be used for computation, yet serve a complete different purpose. Such path could lie into the realm of art. Using coding tools to create art is not a new concept at all. Many artists today embrace code in their practice, creating tools and environments to create interesting pieces of work. For example, Sonic Pi is

an entire coding suite built over SuperCollider sound synthesis server that allows users to create music using code with a superset of Ruby tailored for live coding performances and initially conceived as a programming teaching tool for children in England [1].

Creative computing is a field that has been explored for decades, with many artists and programmers creating tools and environments to create art using code. We may be able to harness some of the principles of creative computing to shape new paradigms of computing that are more accessible and approachable to a wider audience.

The question that orients this work is whether it is possible to design a programming language that (1) materialises programs on a medium radically different from flat text, (2) serves and preserves the computational functionality and formal properties expected of a language, and (3) provides an ecosystem of creative tools that lets the user explore and interact with computation in a free and expressive way. The hypothesis is, then, that *aesthetic expression can serve as a legitimate vehicle for computation.* Assembling tangible pieces can be, simultaneously, a sculptural act and an act of computing.

SCuLPT (*Simple Cubic Language for Programming Tasks*) is the implementation of that hypothesis. It is an artistic sandbox with rules that ensure correct computation, shifting the focus away from computation, yet relying heavily on it to create physical sculptures that, by design, compile to valid programs. Alongside the physical sculptures, the SCuLPT ecosystem includes an environment and runtime to write, test and run programs before assembling the final sculpture. The main contribution is not strictly technical but conceptual: showing that *tangibility*, *aesthetic freedom* and *computation* can coexist in a single language without any of the three being sacrificed.

Chapter 2

Problem Statement

Programming, as it is taught and practiced, excludes those who do not fit its dominant mould. This exclusion breaks down into three interrelated tensions that are worth examining separately.

2.1 Interface Accessibility

The keyboard and the screen are historically situated artefacts. They were designed for users with binaural manual dexterity, classical vision and sustained fine motor control. Any deviation from that profile, such as the temporary or permanent loss of mobility in an arm, dyslexia, low vision, early childhood, or neurodivergence, turns the interface into an active barrier [10, 13]. While disciplines such as music or painting have developed multiple modes of access over centuries, programming has clung to a single channel which, even though it offers accessibility solutions, is not intrinsically inclusive.

2.2 Cognitive Entry Threshold

Learning to program requires mastering an exotic syntax, an abstract semantics and a mental model of execution that the beginner cannot directly inspect. Systematic studies of the learning process [3, 17] document consistent patterns of difficulty across institutions. The consequence is a *filter* that operates before the learner can even evaluate whether programming is of interest to them.

2.3 Paradigmatic Monotony

Most contemporary languages descend, with surface variations, from the same syntactic lineage. The proposals that challenged this precept, such as visual, block-based or end-user programming languages, improved textual accessibility but, for the most part, preserved the fundamental assumption that the program is inscribed on a flat two-dimensional surface. Carrying programming into three-dimensional space, into touch and into the manipulation of objects, remains under-explored, despite robust evidence on the benefits of manipulative learning in other disciplines [12, 11].

2.4 Research Question

From the three tensions above we formulate the guiding question: *is it possible to design a tangible, Turing-complete and aesthetically open programming language that serves as an entry point to computation for more diverse audiences, without sacrificing the formal rigour on which any conventional language depends?* The rest of this document develops an affirmative answer while taking different perspectives and alternatives into account.

Chapter 3

Related Work

SCuLPT is inserted into a conversation that has been going on for several decades about how to open programming to non-specialist audiences. That conversation has produced three families of languages worth reviewing separately, since each one partially resolves the problem SCuLPT poses. We name these families *visual and block-based languages*, *tangible languages for computing education*, and *computing as artistic practice* (*creative computing*). We close the chapter by positioning SCuLPT with respect to the three.

3.1 Visual and Block-based Languages

The line of block-based languages begins with *LogoBlocks* [2] in 1996 and consolidates with *Scratch* [16], developed at the MIT Media Lab's *Lifelong Kindergarten Group*. Scratch replaces textual syntax with draggable blocks whose shape defines what fits with what. This opens an interesting exploration of syntax: in this kind of language, syntax takes on a geometric dimension. The sets of rules that govern program construction become physical rules. That idea has been extended by *Blockly* [6], a Google library that generates code in target languages (JavaScript, Python, Dart) from block programs, and by *Snap!* [7], a reimplementation of Scratch with first-class functions and customisable blocks, aimed at university education.

These proposals drastically reduce syntactic friction for beginners and are now the tool of choice for early computing education. Their limitations, however, are consistent with the diagnosis from the previous chapter. The blocks live on a screen, depend on a keyboard (usually) and a mouse,

and, by design, do not admit meaningful aesthetic freedom beyond changing background colours. The interface has been simplified and computation improved, but its material support has not changed.

3.2 Tangible Languages for Education

A second family moves programming into the physical world. *AlgoBlock* [20] was one of the first systems to propose physical, connectable blocks as a medium for building programs, mainly for collaborative purposes among children. *Tern* [8] extended the idea with wooden pieces recognised by computer vision, with no need for embedded electronics. More recently, *KIBO* [19] combines wooden blocks with barcodes read by a mobile robot, and *Cubetto* [14], commercialised in multiple markets, uses a grid and magnetic tiles to program the movement of a small robot. *Osmo Coding* [21] follows a hybrid line: the pieces are physical but execution happens on a tablet that observes them through the camera.

These systems amply demonstrate that tangibility reduces cognitive and motivational barriers for student audiences, in line with prior literature on *tangible user interfaces* [9, 11]. They share, however, three recurring limitations. First, they are all designed as *educational tools*: their expressive power is intentionally bounded and applied to a specific domain, usually robotics. This provides a more concrete, manipulative and stimulating programming experience for students, but the resulting systems are still domain-specific languages, or rather, programmable didactic tools. Second, the pieces are closed. They come in a shape, colour and size preset by the manufacturer, and the user usually cannot modify them without destroying or altering their functionality. That freedom, both aesthetic and functional, is absent by design. The drive to control the user experience is understandable in an educational context, but it limits expressiveness and the range of users who can use these systems. The freedom to modify pieces matters for users with accessibility needs, who may require pieces of a particular size, material or morphology to manipulate them. Third, most of these systems come with a price tag that not every educational institution or individual can afford. In addition to needing specialised hardware (and sometimes software), the cost of the pieces, their maintenance and access can be a barrier to adoption. This is especially relevant in low-resource contexts, where access to educational tools is crucial to close

learning gaps.

3.3 Computing as Artistic Practice

A third tradition approaches the problem from a different angle. It keeps the textual interface of programming but aims it explicitly at artistic production. *Processing* [15] offers a Java dialect intended for visual artists and designers, and popularised the idea that writing code is a legitimate form of artistic practice. *Sonic Pi* [1] builds on the SuperCollider synthesis engine a Ruby superset aimed at musical *live coding*, originally conceived as an educational tool for schools in England. The practice of *live coding*, both musical and visual, has been systematised as an artistic-computational movement in its own right [4], with dedicated festivals and communities (*e.g., Algorave*).

These initiatives have shown that programming *can* be art and that there is a trained and enthusiastic audience for that vision. They share, however, a structural limitation with the first family: they do not abandon the typewriter paradigm. The *live coding* artist still types on a screen in front of an audience (and gets stressed when the code does not compile); the final artistic output is sound, video or image, not the physical body of the program.

3.4 Positioning of SCuLPT

SCuLPT sits at the seldom-travelled intersection of the three preceding families. It shares with block-based languages the idea that syntax can be a geometric property of the pieces, essential to building programs. The natural intuitiveness of fitting pieces together is a powerful resource for assimilating complex syntactic concepts and grounds them in a concrete, understandable construction logic: pieces either fit or they do not. It shares with tangible languages the bet on moving programming off the screen and into three-dimensional space. Enabling the world to program necessarily implies opening programming to forms of access other than the traditional one. The aim is to leverage the manipulative and open nature of the pieces to open programming to users with diverse capabilities, resources and accessibility needs. Finally, it shares with the *creative computing* tradition the conviction that artistic practice is a legitimate vehicle for

computation, not a later ornament. Table 3.1 summarises the differences.

Table 3.1: Comparison of SCuLPT with related families.

	Block/visual	Tangible edu.	Creative comp.	SCuLPT
Physical support	No	Yes	No	Yes
Turing-complete	Partial	No	Yes	Yes
Aesthetic freedom	Low	Low	Medium	Very high
Target audience	Students/Novices	Children	Artists	Mixed

To the best of our knowledge, SCuLPT is the first language to combine *physical tangibility*, *formally proven Turing-completeness* and *radical aesthetic freedom* in a single design. The radicalness of the aesthetic openness deserves to be underlined: in SCuLPT the user can freely modify the shape, size, colour, material and texture of the pieces; the only constrained property is the topology of the connections. A piece may be a standard PLA cube or an animal figure carved in wood, and the program will work the same as long as the connection points respect the specification.

Chapter 4

Objectives

In the creation of SCuLPT we posit two objectives. First, we require to build a fully capable physical programming language addressing the concerns described previously about the state of programming, computation and programming languages. SCuLPT is designed to be a visual and physical oriented programming language, with a heavy focus on the ease of use, computational correctness and aesthetics. SCuLPT is designed to be a Turing complete language with a very small instruction set, allowing users to create any program they desire.

Second, we aim to evaluate the language in two main aspects. First, we intend to evaluate the ease of the language to foster computational thinking without the intrinsic difficulties of current programming languages imposed by the current interfaces. This is done by evaluating the performance of users with different backgrounds and levels of experience in programming, from novices to expert programmers, in solving small computational problems using SCuLPT and their overall experience with the language. Second, we evaluate the properties of SCuLPT as a paradigm that breaks existing preconceptions about programming languages and moves beyond the typewriter interface by leveraging artistic creativity as an alternative. This is done by letting users create pieces freely using SCuLPT, allowing them to explore the language and its components without any pressure to complete any tasks. This allows us to evaluate the overall experience of using SCuLPT as an artistic tool, as well as any suggestions for improvement from an aesthetic lens.

Chapter 5

Methodology

5.1 Design and Implementation

SCuLPT is a visual and physical oriented programming language that aims to lower the entry threshold to programming by leveraging artistic creativity as an alternative to the traditional typewriter interface. SCuLPT as a whole is in fact two interconnected languages, one being the visual language and specification that defines the components from a strictly physical standpoint and the other being the programming language that is used to define the behaviour of each component. For the development of SCuLPT, we have followed a user-centred design approach, creating a visual language that is both usable and accessible to a wide range of users with the possibility of modifying the language to suit their needs. SCuLPT is focused on usability and accessibility is displayed by using simple but distinctive shapes and forms for each component on the language. This first design choice allows for users to easily identify and differentiate between different components of the language, making it trivial to convey meaning behind each shape while allowing people with any background to recognize each component foregoing knowledge, language or capacity barriers that arise from conventional programming languages.

Regarding the programming language, SCuLPT is designed to be a Turing complete language with a very small instruction set of only 13 instructions. By building structures and following the intuitive rules of the physical language, users can create any program they desire. The language is designed to be simple and easy to understand, with a focus on the visual representation of the program rather than the code itself. Code block are physically assembled together to create a

program, with each block representing a different operation. Blocks are designed such that their connections are intuitive and easy to understand, with each block having a set of features that must be connected to other blocks in order to function correctly. This allows users to create programs without the need to understand complex syntax or semantics, making it accessible to users with little to no programming experience.

Regarding the language's aesthetics, SCuLPT is designed to be an artistic sandbox. The rules and specification for how shapes should look and feel are very loose. This allows users to tinker with the components, suiting them for their needs and desires. The only rules that must be followed only consider component connections and a set of features each block requires. Final shape, size, material, colour or texture are completely open for anyone to modify without any impact on the program. This provides a sense of freedom and creativity that is not present in other programming languages, allowing users to express themselves through their code and create unique and personal pieces. This also implies that the language can be fitted to almost anyone's needs, allowing accessibility and customization for users with different physical or cognitive capacities or context where SCuLPT will be used.

SCuLPT current implementation used for testing features small 3D printed components in polylactic acid (PLA) with varying colours. The printed pieces follow the standard SCuLPT implementation with no modifications to the shapes. The components are connected using magnets and screw elbow joints, allowing for easy assembly and disassembly of the components, making it easy to create and modify programs on the fly.

SCuLPT has a double-edge sword, as it is strictly physical media, execution is tied to the interpreter, a human interpreter. Human interpretation is one of the main aspects of the language, yet humans are not perfect machines. This implies that correct computation is not guaranteed. Human interpretation is part of SCuLPT's exercise as a *performative act*: walking through the pieces and applying instructions is itself an embodied gesture that belongs to the language, not a step external to it. This performative dimension is also what makes the interpretation unreliable. Errors in the program's execution are bound to happen at some point and the language is not designed to be fault tolerant from a strict computation standpoint. To overcome this, SCuLPT also features the Simple Cubic Language for Programming Task's Environment and Runtime, SCuLPTER for short. Written in Scala, SCuLPTER is a transpiler written in Scala

capable of taking a text representing a SCuLPT program and running it step by step to allow for debugging and testing of the program before it is physically assembled. SCuLPTER is designed to be lightweight and portable, allowing it to run in any browser thanks to the Scala.js web framework. This makes easily distributable and accessible to users wanting to have a reliable method of interpreting their sculptures.

5.2 Validation

Regarding validation, we have performed a series of sessions with users from different backgrounds and levels of experience with programming. For our sessions, users ranged from novices with no prior programming experience to expert programmers with years of experience in the field. Users also varied in ages, ranging from secondary school students to graduate students and professionals. Tests were also performed with users with little to no knowledge in programming, but rather in art related fields. The tests were designed to evaluate the usability and accessibility of SCuLPT, as well as the overall experience of using the language.

This sessions consisted of several moments, starting with a brief introduction (30-45 minutes) to the language and its components. This introduction was tailored to each group of users, with novices receiving a more in-depth explanation of the language and its physical components, while expert programmers received a more technical overview of the language and its computational features. Afterwards, users were given the opportunity to explore the language and its components (1 hour), with a focus on the physicality of the blocks and their connections. Growing familiarity with the language and its components was the main goal of this section, allowing users to explore the language and its components without any pressure to complete any tasks. Next, users were given a set of tasks to complete using SCuLPT, with the goal of evaluating the usability and accessibility of the language. Several sets of tasks were created varying in complexity and difficulty to accommodate the different levels of experience of the users. This tasks consisted of simple programs that could be created using the language, such as creating a simple loop to add numbers or conditional statements to break from such loops.

During this test phase (2 hours), users were allowed to ask questions and receive help from the facilitators, but were encouraged to complete the tasks on their own. User were also encouraged

to explore the language and its components, with the goal of creating a program that was both functional and visually interesting. Finally, users were given a survey to evaluate their experience with SCuLPT, with a focus on the usability, accessibility of the language, and their overall programming knowledge. In the special case of underage students, the accompanying teachers were asked to fill the survey as well. The survey also contained questions focused towards the educators themselves, such as their overall reception of the language and its components, as well as their thoughts on the potential use of SCuLPT in their classrooms. The survey was designed to gather feedback on the overall experience of using SCuLPT, as well as any suggestions for improvement from a technical and aesthetic lens.

Evaluation of the results was done using a combination of qualitative and quantitative methods. Qualitative methods included user interviews and surveys, while quantitative methods included measuring the time taken to complete each task and the number of errors made during the process. Interviews were conducted after the tests to gather feedback on the overall experience of using SCuLPT with some individuals, as well as any suggestions for improvement from a technical view. The results of the tests were analysed to determine the overall usability and accessibility of SCuLPT, as well as the overall experience of using the language. Additional tests were performed to evaluate the overall experience of using SCuLPT as an artistic tool, with users being asked to create a piece of art using the language. The results of these tests were also analysed to determine the overall experience of using SCuLPT as an artistic tool, as well as any suggestions for improvement from an aesthetic lens.

Chapter 6

Results

6.1 Language Implementation

SCuLPT as a programming language is a functioning language capable of writing any program.

The language uses FILO (First In Last Out) stacks as its only data structure to store and manipulate data. Each stack is represented by a different shape other than the reserved shapes used by the instructions, and they are initialized on first call. Stacks can hold an arbitrarily large amount of rational numbers. Stacks allow the pop, push, move and duplicate operations. Numbers are defined by their literal counterparts, and the language supports basic arithmetic. Regarding their representation, stacks are represented by any shape that is not reserved for the instructions,, such as simple shapes like circles, triangles or squares or complex drawings. Anything goes.

The six arithmetic operations—addition, subtraction, multiplication, division, modulo and comparison—are the only operations available in SCuLPT. Each operation exists in two forms, distinguished by the number of parameters the block receives, which corresponds to physically different blocks:

- **Unary** (one parameter: a stack). The top two elements of the given stack are popped, the operation is applied, and the result is pushed back onto the same stack. For example, `add stack` pops the top two values and pushes their sum.
- **Binary** (two parameters: a stack and a number literal). The top element of the given

stack is popped, the operation is applied with the number literal as the second operand, and the result is pushed back onto the same stack. For example, `add stack 3` pops the top value and pushes its sum with 3.

For non-commutative operations (subtraction, division and modulo), the first operand is always treated as the left-hand side and the second as the right-hand side; for instance, `sub stack 3` computes $top\_of\_stack - 3$. Comparison (`cmp`) follows the same unary and binary pattern: it pushes 1 if the left operand is greater than the right, 0 if they are equal, and -1 if the left is less than the right.

In cases where the arithmetic is not defined, such as division or modulo by zero, a *nil* value is pushed onto the stack. When a *nil* value is encountered in an arithmetic operation, other than comparison, the program halts in an error state.

Comparison interacts uniformly with the special value `nil`: when applied to an empty stack, to a stack with only one element, or with any operand equal to `nil`, the result is `nil`. The question operator `?` treats as the "false" branch (*i.e.,* skips the next instruction).

SCuLPT allows for flow control through the use of loops, conditional blocks and jump blocks. Loops are physically represented by a literal loop of connected blocks or a single block jump with a negative number as a parameter. The jump block is a special block that allows for the execution to continue at any point by moving the execution from the current block to another with the offset given by parameter. Conditional statements are represented by a question block using a stack value as a parameter. Positive values are considered true and evaluate the following block, and negative values are considered false and do not evaluate the affected block, but rather skips the next instruction.

SCuLPT is based on the physical blocks. Blocks as three-dimensional objects are unwieldy to be described in text. Therefore, we have made several valid notations for SCuLPT programs. Every block is represented as a square with the shape of each block inside the square. The connections between blocks are represented by arrows between the squares. As a first example and introduction to the language itself, we have implemented a simple program that calculates the Fibonacci sequence in SCuLPT.

The program in Figure 6.1 fills the circle stack with the numbers of the Fibonacci sequence.

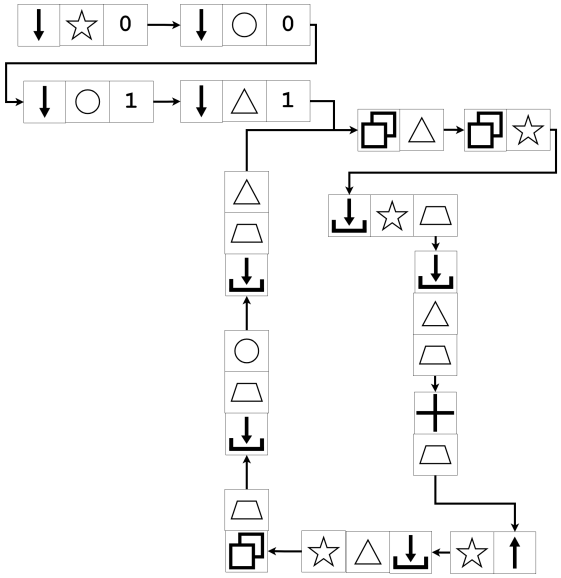

Figure 6.1: SCuLPT Fibonacci program

The second notation is a more traditional representation of code, which is used by the SCuLPTER interpreter, which will be explained afterwards. The program is written in a more traditional way, every block has a reserved name, connections are omitted. It reads as a list of instructions, somewhat similar to an assembly language. Limitations of this implementation are that the program is not visually represented, connections between blocks are not shown and physical loops cannot be represented, forcing the user to only use jump operations for execution control. Our Fibonacci implementation can be rewritten in SCuLPTER as follows.

Algorithm 1 Fibonacci sequence in SCuLPT

push 0 **star**

push 0 **circle**

push 1 **circle**

push 1 **triangle**

dup **triangle**

dup **star**

mov **trapezoid star**

mov **trapezoid triangle**

add **trapezoid**

pop **star**

mov **star triangle**

dup **trapezoid**

mov **circle trapezoid**

mov **triangle trapezoid**

jmp **-10**

Writing a Fibonacci program is a simple task in SCuLPT which gives us a good insight on how SCuLPT looks and feels, yet it is not a complete proof of the language's capabilities. The language is Turing complete and it is capable of writing any program that can be written in any other programming language.

6.2 Turing Completeness

We now prove that SCuLPT is Turing complete by showing that any Turing machine can be simulated by a SCuLPT program. The proof proceeds by constructing a translation Φ that maps an arbitrary Turing machine M to a SCuLPT program $\Phi(M)$, then showing that $\Phi(M)$ faithfully simulates M step by step.

6.2.1 Preliminaries

Turing Machine. A Turing machine is defined as $M = (Q, \Gamma, b, \Sigma, \delta, q_0, F)$ where Q is the finite set of states, Γ the tape alphabet, $b \in \Gamma$ the blank symbol, $\Sigma \subseteq \Gamma \setminus \{b\}$ the input alphabet, $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ the transition function, $q_0 \in Q$ the initial state, and $F \subseteq Q$ the set of halting (accepting) states.

Instruction Reference. For the purposes of this proof, the relevant SCuLPT instructions have the following semantics. All stacks are FILO and hold rational numbers; any identifier not reserved for an instruction names a stack, created on first use and unbounded in size.

- `push` n S: Push rational number n onto stack S.
- `pop` S: Remove the top element of S. No-op if S is empty.
- `mov` S_1 S_2: Pop the top of S_2 and push it onto S_1 (S_1 is the destination, S_2 the source). If S_2 is empty, push `nil` onto S_1.
- `dup` S: Duplicate the top of S. If S is empty, push `nil`.
- `cmp` S: Pop the top two elements a, b of S; push 0 if $a = b$, -1 if $a < b$, 1 if $a > b$. If either operand is `nil`, or if the stack has fewer than two elements, push `nil`.
- `cmp` S n: Replace the top element a of S with 0 if $a = n$, -1 if $a < n$, 1 if $a > n$. If a is `nil` or S is empty, push `nil`.
- `?` S: Pop the top of S. If the value is ≥ 0, execute the next instruction; if the value is < 0 (or nil), skip the next instruction.
- `jmp` k: Move the program counter by relative offset k.
- `neg` S: Negate the top of S in place.
- `mul` S: Pop the top two elements, push their product.

Blank-Symbol Convention. The special value `nil` encodes the blank tape symbol b. Operations on empty stacks produce `nil` wherever a value is needed: `dup` on an empty stack pushes `nil`; `mov` from an empty source pushes `nil` onto the destination; `cmp` with any `nil` operand (*e.g.,* an empty stack) yields `nil`. The question operator `?` treats `nil` as a "false" branch and skips the next instruction.

6.2.2 Encoding

Tape Encoding via Two Stacks. The tape is represented by two named stacks, `left` and `right`. The top of `right` encodes the current cell under the head (or blank if empty). The top of `left` encodes the cell immediately to the left of the head. Tape symbols are encoded as follows: $b \mapsto$ `nil`, $\sigma_1 \mapsto 1$, $\sigma_2 \mapsto 2$, etc. The non-blank symbols are positive integers; the blank is the dedicated `nil` value, which arises naturally from operations on empty stacks.

An auxiliary stack `aux` is used for non-destructive symbol comparison during state transitions. It is empty between transitions and does not encode any part of the tape.

State Encoding. Each state $q \in Q$ maps to a contiguous block of instructions $B(q)$. Halting states map to the `HALT` block, which simply terminates execution.

Blank Tape Initialization. For input $w = w_1 \dots w_n$, the `INIT` block pushes $\text{enc}(w_n), \dots, \text{enc}(w_1)$ onto `right` (so the top is $\text{enc}(w_1)$). For a blank initial tape, `INIT` is empty.

6.2.3 The Translation Φ

Given a Turing machine $M = (Q, \Gamma, b, \Sigma, \delta, q_0, F)$, the translation $\Phi(M)$ produces a SCuLPT program with the following structure:

```
[ INIT block ]
[ B(q0) ]
[ B(q1) ]
  ⋮
[ B(qn) ]
[ HALT block ]
```

For each non-halting state $q \in Q$, the code block $B(q)$ consists of three phases.

Phase 1: Read Current Cell (2 instructions).

`dup right`	▷ copy top of `right`; pushes `nil` if empty (blank)
`mov aux right`	▷ transfer copy to `aux`; `right` restored

After Phase 1, `aux` $= [v]$ where $v =$ enc(current cell) (or $v =$ `nil` for blank), and `right` is unchanged.

Phase 2: Symbol Dispatch Chain ($7|\Gamma|$ instructions). For each symbol $\sigma_i \in \Gamma$, one block of 7 instructions performs an equality test:

`dup aux`	▷ `aux` $= [v, v]$
`cmp aux` σ_i	▷ `aux` $= [\mathrm{cmp}(v, \sigma_i),\ v]$; binary cmp replaces top
`dup aux`	▷ `aux` $= [r, r, v]$ where $r = \mathrm{cmp}(v, \sigma_i)$
`mul aux`	▷ `aux` $= [r^2, v]$; $r^2 \in \{0, 1\}$
`neg aux`	▷ `aux` $= [-r^2, v]$; 0 iff equal, -1 iff not
`? aux`	▷ pops $-r^2$; execute next if $= 0$, skip if $= -1$
`jmp` *offset*(σ_i)	▷ jump to handler (reached only on match)

Correctness. If $v = \mathrm{enc}(\sigma_i)$: cmp returns 0, $0^2 = 0$, $-0 = 0$, and `?` on 0 executes (since $0 \geq 0$), so the jump is taken. If $v \neq \mathrm{enc}(\sigma_i)$: cmp returns ± 1, $(\pm 1)^2 = 1$, $-1 < 0$, and `?` skips the jump. After `?` pops, `aux` $= [v]$, preserving the cell value for the next comparison.

Phase 3: Symbol Handlers ($5|\Gamma|$ instructions). For each $\delta(q, \sigma_i) = (q', \sigma_i', D)$, the handler writes the new symbol and moves the head:

`pop aux`	▷ discard cell value copy; `aux` $= \emptyset$
`pop right`	▷ remove current cell
`push` σ_i' `right`	▷ write new symbol

If $D = R$ (move right):

`mov left right`	▷ pop σ_i' from `right`, push onto `left`
`jmp` *offset*(q')	▷ jump to $B(q')$

If $D = L$ (move left):

`mov right left`	▷ pop `left`'s top, push onto `right`; if empty, pushes `nil`
`jmp` *offset*(q')	▷ jump to $B(q')$

Implicit Blank Extension.

Lemma 6.1. *No explicit blank-extension code is required in any state block $B(q)$.*

Proof. The argument rests on two facts: (a) operations on empty stacks (`dup`, `mov`, `cmp`) produce `nil`; (b) the encoding identifies the blank symbol b with `nil`, *i.e.,* $\text{enc}(b) = \texttt{nil}$. Hence an empty stack already represents an unbounded suffix of blank cells. Concretely:

- Phase 1: `dup right` on an empty `right` pushes `nil`. `mov aux right` transfers this `nil` to `aux`, restoring `right` to empty. By the encoding, the value read is $\text{enc}(b)$, as required.
- Phase 2: when $v = \texttt{nil}$, `cmp aux` σ_i yields `nil` for every non-blank σ_i (since σ_i encodes a positive integer, never `nil`); subsequent `mul` and `neg` propagate `nil`, and `?` on `nil` skips the jump. The blank dispatches uniquely to the handler reserved for b via a direct `nil`-aware test, or by being the only symbol that fails every non-blank comparison and falls through to a default handler. The chain therefore correctly identifies the blank without consulting an explicit cell value.
- Phase 3, $D = R$: After `mov left right`, if `right` is now empty, the next Phase 1 reads `nil`, which the encoding identifies with b.
- Phase 3, $D = L$: `mov right left` on an empty `left` pushes `nil` onto `right`, which again represents b by the encoding.

At no point does the construction depend on explicit blank-pushing code: blanks emerge as the natural value of operations on empty stacks, and the question operator `?` routes `nil` through the program just as it routes negative numbers. Each $B(q)$ therefore has a fixed size regardless of tape boundaries. □

Block Size and Jump Offsets. Each $B(q)$ has $|B(q)| = 2 + 12|\Gamma|$ instructions (2 for Phase 1, $7|\Gamma|$ for Phase 2, $5|\Gamma|$ for Phase 3), constant across all states.

Let $C = 2 + 12|\Gamma|$ and $I = |\text{INIT}|$. The jump offset from Phase 2's σ_i-block to its Phase 3 handler is:

$$\mathit{offset}(\sigma_i) = 7|\Gamma| - 2i - 6$$

The cross-block offset from the handler for σ_i in $B(q_j)$ to $B(q_{j'})$ is:

$$\mathit{offset}(q') = (j' - j)\, C - (7|\Gamma| + 5i + 6)$$

All offsets are closed-form and computable at translation time.

6.2.4 Correctness

Simulation Invariant. Define the SCuLPT program state to be *k-consistent with M on input w* if, after simulating k steps of M:

1. `aux` is empty.
2. The top of `right` (treating empty as `nil` = enc(b)) equals enc($v_k[0]$), the encoding of the current cell.
3. The remaining elements of `right` encode cells to the right of the head.
4. The elements of `left` encode cells to the left (top = immediately left).
5. The program counter points to the first instruction of $B(q_k)$.

Theorem 6.1 (Halting Preservation)**.** *If M halts on input w after n steps, then $\Phi(M)$ reaches the `HALT` block and terminates.*

Theorem 6.2 (Non-Termination Preservation)**.** *If M does not halt on input w, then $\Phi(M)$ never reaches the `HALT` block and does not terminate.*

Proof of Theorems 6.1 and 6.2. We prove by induction on the computation step k that the simulation invariant is maintained.

Base case ($k = 0$). `INIT` pushes the input encoding onto `right`. Stacks `left` and `aux` are empty. The program counter is at $B(q_0)$. The invariant holds by construction.

Inductive step. Assume k-consistency. Let $\delta(q_k, v_k[0]) = (q_{k+1}, \sigma', D)$.

Phase 1: `dup right` followed by `mov aux right` places enc($v_k[0]$) on `aux` (or `nil` if `right` is empty). `right` is restored.

Phase 2: The dispatch chain tests each $\sigma_i \in \Gamma$. For $\sigma_i = v_k[0]$, the equality test succeeds (cmp returns 0, $0^2 = 0$, neg(0) = 0, `?` on 0 executes, jump taken). For $\sigma_i \neq v_k[0]$, the test fails (cmp returns ± 1, squared gives 1, negated gives -1, `?` skips). Each failed test leaves `aux` = [enc($v_k[0]$)].

Phase 3: `pop aux` empties `aux` (condition 1 restored). `pop right` removes the current cell. `push` σ' `right` writes the new symbol.

Case $D = R$: `mov left right` pops σ' from `right` and pushes it onto `left`. Now `left`'s top is σ' (the written cell, to the left of the new head position) and `right`'s top is the next cell to the right (or empty, representing blank).

Case $D = L$: `mov right left` pops the top of `left` and pushes it onto `right`. If `left` is empty, it pushes `nil` (blank). Now `right`'s top is the cell to the left (or blank), correctly representing the new head position.

In both cases, `jmp` *offset*(q') sets the program counter to $B(q_{k+1})$, establishing $(k+1)$-consistency.

By induction, the invariant holds for all k. If M enters a halting state at step n, the jump targets the `HALT` block and execution terminates (Theorem 6.1). If M never halts, each state block executes a bounded non-zero number of instructions and transfers control to the next, so execution continues without bound (Theorem 6.2). □

6.2.5 Generality and Complexity

The translation Φ is total (defined for every TM), computable, and produces programs of size $|\Phi(M)| = O(|Q| \times |\Gamma|)$. Since every computable function can be computed by some Turing machine, and Φ faithfully simulates any such machine in SCuLPT, we conclude:

Theorem 6.3. *SCuLPT is Turing complete.* □

6.2.6 Worked Example: Busy Beaver Simulation

As a concrete demonstration that SCuLPT can simulate Turing machines, we exhibit an implementation of the 3-state Busy Beaver machine (BB_3). This machine uses states $\{A, B, C\}$, tape

alphabet $\{b, 1\}$ with blank symbol b (encoded as $b \mapsto$ `nil`), and is known to write six 1s on the tape before halting after 14 steps.

The implementation uses a 4-stack encoding (`left`, `right`, `head`, `temp`) that stores the current cell in a dedicated `head` stack, an alternative to the 2-stack encoding used in the general proof. Each state block reads the current symbol from `head`, branches on its value using `cmp` and `?`, writes the new symbol, and moves the head by transferring values between `left`, `right`, and `head` via `mov`. Its correctness follows from the same underlying principle: SCuLPT's named FILO stacks, conditional branching, and relative jumps suffice to encode arbitrary Turing machine configurations and transitions.

Algorithm 2 3-State Busy Beaver Turing machine in SCuLPT

push nil **left**
push nil **left**
push nil **left**
push nil **left**
push nil **right**
push nil **right**
push nil **right**
push nil **right**
push nil **head**
// STATE A
push 1 **temp**
mov **temp head**
cmp **temp**
? **temp**
jmp 13 **// Go to A_1**
push 1 **head**
mov **right head**
mov **head left**
// STATE B
push 1 **temp**
mov **temp head**
cmp **temp**
? **temp**
jmp 9 **// Go to B_1**
push 1 **head**
mov **left head**
mov **head right**
jmp − 16
// A_1
push 1 **head**
mov **left head**
mov **head right**
jmp 5 **// Go to C**
// B_1
push 1 **head**
mov **right head**
mov **head left**
jmp − 16 **// Go to B**
// STATE_C
push 1 **temp**

The general proof via Φ subsumes this example: $\Phi(BB_3)$ is a specific instance of the universal construction, producing a program of $2 + 12 \cdot 2 = 26$ instructions per state block (78 total for 3 states, plus initialization and halt), and its correctness follows immediately from Theorems 6.1 and 6.2.

The complete implementation of SCuLPT is open source and is available on GitHub[1].

6.3 Interpreter Implementation

SCuLPTER is an interpreter for SCuLPT built in Scala that allows users to write and execute SCuLPT programs in a more traditional way. It leverages the Scala.js framework to provide a web-based interface for users. This makes the interpreter lightweight, distributable and easy to use without the need for any additional software. SCuLPTER features a simple text editor where a program can be written. It also features buttons to lex, parse and execute the program. Execution of the program is done in a step-by-step manner, allowing users to see the state of the program at each step. SCuLPTER code is open source and is also available on GitHub[2].

6.4 Validation

To validate SCuLPT, we have conducted a series of sessions with different users to test the language and its ecosystem. A total of 6 sessions were conducted with a total of 75 users, ranging from novices with no prior programming experience to expert programmers with years of experience in the field. The sessions were designed to evaluate the usability and accessibility of SCuLPT, as well as the overall user experience of using the language.

6.4.1 Regarding Usability

Usability was evaluated through questions in the survey given to users after the sessions, as well as through user interviews. The results of the usability questionnaire showed that users found SCuLPT, from a physical standpoint, to be easy to use and understand, with a majority of

[1] `https://github.com/Darkoyd/SCuLPT`
[2] `https://github.com/Darkoyd/SCuLPTER`

users reporting that they were able to use and understand the shapes and blocks of the language with relative ease. Among the users, especially the younger demographics, the physicality of the blocks was a key factor in their understanding of the language.

One concern that was raised by the researchers was that of overall piece integrity and the possibility of losing pieces. Continuous use of the blocks in a classroom setting could lead to pieces being lost or damaged, which could hinder the usability of the language. PLA is a material notorious for being a somewhat soft plastic prone to deformation and damage, especially with repeated use. As expected, some pieces were lost during the sessions, but the overall integrity of the blocks was maintained. The blocks were able to withstand the repeated use and manipulation by the users. The remaining pieces were still usable and intact, and the users were able to continue using the language without any issues.

These results shows that SCuLPT as a building block construct and visual language performs as expected, with users sharing overall positive feedback regarding the usability of the language.

6.4.2 Regarding Computation

Another section of the survey was dedicated to evaluate, from a more technical eye, the ease of use of SCuLPT as a programming language. Alongside the survey, users were given a set of programming tasks to complete using SCuLPT. The tasks were designed to test the users' understanding of the language and its capabilities, as well as their ability to write programs in SCuLPT. Throughout the sessions, about 50% of the total participants were able to complete the tasks. Per sub-population, however, the picture is not the one one might expect. Table 6.1 summarises the findings by group.

The most unexpected result was the difficulty experienced by expert programmers: despite their prior experience, they did not outperform novice teenagers in task completion. A plausible reading is that the mental model afforded by experience with textual languages does not transfer directly to reasoning over LIFO stacks assembled physically. Teenagers, without those prior models, approached the language on its own terms.

Table 6.1: Findings on the computational tasks by sub-population.

Sub-population	Completion	Qualitative observation
Children	Low	Excellent physical handling; conceptual difficulty.
Novice teenagers	Medium	Completed in ∼1.5 h on average.
Novice adults	Low	Understood the language without completing tasks.
Expert programmers	Low	Difficulty similar to novices.
Artists	None	Understand the pieces, do not complete tasks.

Survey data

The quantitative survey (Likert scale 1–5) was administered to a sub-set of $n = 25$ voluntary participants for whom complete responses were obtained. The questions were grouped into five blocks: (1) demographic information and prior experience, (2) quantitative aspects of the language (*e.g.,* difficulty, clarity), (3) qualitative aspects of the language (*e.g.,* enjoyment, creativity, perceived usefulness), (4) comparative evaluation of programming knowledge, and (5) educators' perception of SCuLPT's pedagogical potential. Table 6.2 summarises the means ranges per question block.

Table 6.2: Aggregates of the quantitative survey ($n = 25$, scale 1–5).

Question block	Mean (range)
Quantitative aspects of the language	2.2 – 3.4
Comparative evaluation against textual languages	2.8 – 3.5
Educator/evaluator perception	4.0 – 4.3

The difference between blocks is informative. Participants' technical perception is medium-low, consistent with the reported difficulty completing the tasks. The perception of the teachers who accompanied the sessions is clearly positive, which suggests pedagogical viability even when learners' immediate performance is modest. Qualitative responses reinforce both sides: participants highlight that "bringing programming into the 3D world is a feat" and that the language is "interesting and creative for young children", but they also report that "the blocks sometimes

generate confusion" and that "the operation concepts are confusing". The most frequent suggestions point to supporting didactic material (slides, incremental exercises) and to integration with concrete domains such as video games or mathematics.

Results show that SCuLPT as a programming language can be easy to understand and use, yet it is not easy to write programs in SCuLPT. The language is capable of writing any program, but it requires a certain level of understanding of the language and its capabilities. This is a common issue with programming languages that SCuLPT tried to eliminate, but it is still present.

6.4.3 Regarding Aesthetics

Special sessions were held with a group of artists to test their overall experience with SCuLPT from their experience as artists. Results were surprisingly positive, with most users reporting that they enjoyed very much the experience of using SCuLPT. Participants expressed their appreciation for the physicality of the blocks and the ability to manipulate them in a tangible way. The physicality of the block "was quintessential to their experience". Participants also reported that the blocks were easy to understand and use, and that they enjoyed the process of building their sculptures. When presented with the programming tasks, participants were able to easily understand the assignments, understand the block and their functionalities, but they were not able to complete any task. Nevertheless, they expressed overall satisfaction while trying and failing to complete the tasks. Participants reported that the experience of using SCuLPT was enjoyable and that they would like to use it again in the future. This last point lead to additional sessions with the same group of artists, as they were engaged with SCuLPT and wanted to explore it further. While trying and failing to complete the tasks, they used the blocks to create sculptures that were not intended to be functional, but rather artistic. Participants reported that they enjoyed the process of creating sculptures and that they found it to be a rewarding experience. Surprisingly, participants used the blocks in ways that were not intended from the start. Creating visually interesting constructs with this newfound freedom.

These results paint SCuLPT as a visually interesting, and even fun, medium for creative expression. Although this document has been focused on SCuLPT from a technical standpoint, it is important to note that SCuLPT was not only made to be a programming language, but also

a medium for creative expression.

Chapter 7

Discussion

The results in Chapter 6 paint an ambivalent picture that deserves careful discussion. On one hand, SCuLPT is technically successful: the language is Turing-complete, its interpreter works, the physical pieces withstand use, and most participants identify and manipulate the components without difficulty. On the other hand, writing programs in SCuLPT remains hard. Roughly half of the participants complete the computational tasks, and expert programmers do not outperform novice teenagers on that metric.

This tension is not a failure of the language but one of its constitutive features. SCuLPT does not seek to eliminate the intrinsic difficulty of programming, a problem shared by every language. The intention is to change the *nature* of that difficulty and, in doing so, to broaden who finds it interesting. What the results show is that the difficulty is redistributed: the barrier to first interaction with the language falls sharply, children manipulate pieces and artists enjoy them. What stays almost untouched is the difficulty of reasoning algorithmically about stacks, conditionals and loops. The discrepancy between the low technical perception of participants (Likert means 2.2–3.4) and the high educator perception (4.0–4.3) reinforces this reading. SCuLPT works better as a didactic vehicle mediated by a facilitator than as an autonomous programming tool.

7.1 Physicality and Aesthetics as Objects of Study

Physicality and aesthetics deserve to be treated as objects of study in their own right, not merely as means to an end. The sessions with artists are the clearest case. Participants who reported the greatest aesthetic satisfaction were also those who most pulled the pieces away from their prescribed computational use, building non-functional sculptures and exploring assemblages not anticipated in the specification. In their hands the language stopped being a tool and became a material. This observation connects directly to open questions in the *programming experience* community: what does it feel like to program in a medium that can be held, what creative purposes emerge when code leaves the screen.

7.2 Positioning

SCuLPT does not aspire to replace textual languages. Its cumbersome nature for large software projects is instantly apparent. The size of a program is bounded by the number of pieces that can physically be assembled, the printing cost grows linearly with that number, the assembly takes time, and, being only physical, the assembled result does not run on a computer. SCuLPT functions, instead, as an entry point for audiences historically distant from programming and as an expressive medium for artists who want to incorporate computation into their practice without going through the keyboard.

7.3 Limitations

Beyond the size constraints already mentioned, three further limitations are worth naming. The first and largest is rooted in the physical nature of the exercise. The validation sessions require facilitators who guide the participants, resolve doubts, help assemble sculptures, supervise the use of the pieces and, in the case of minors, make sure that teachers accompany the students. This makes the scalability of the exercise limited, and its application in educational contexts dependent on the availability of trained facilitators.

The second limitation goes hand in hand with the first. The use of 3D-printed pieces with magnetic connectors is a practical solution for prototyping and manufacturing the language at an

accessible cost. The material is fragile and wears down with use, however, reducing the inventory available for each session. The time cost of printing enough pieces to run the workshops is also high given the limited availability of 3D printers. This restricts the number of participants who can be served in a session and is visible in the results: the number of participants is small and the number of computational tasks assigned is limited. The limitation can be overcome with more resistant materials, mass printing processes, and design optimisation.

Third, the validation samples, although diverse, are not large. Educational performance over the longer term, in stable cohorts, remains an open question.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This document presented SCuLPT, a tangible programming language whose pieces are 3D-printed and physically assembled to build Turing-complete sculptures. Over a minimal set of 13 instructions operating on LIFO stacks, the language's completeness was formally proven via a translation Φ that maps arbitrary Turing machines to SCuLPT programs. The aesthetic freedom of the pieces distinguishes the language from the three families of related work and, at the same time, broadens its possibilities for accessibility and expressive use.

The validation with diverse audiences showed a consistent pattern. Physicality works as an entry point and the components are recognisable across audiences, but programming still requires algorithmic reasoning and that does not disappear by changing the medium. The educator perception, more positive than that of the learners, suggests that the pedagogical potential of SCuLPT is greater than its short-term results allow us to measure. Artists, for their part, found in the language an expressive medium in its own right, beyond its computational capacity without leaving it aside. A fuller critical reading of these results, their tensions and their limits is developed in Chapter 7.

8.2 Future Work

Several lines of research remain open regarding the language and its artistic and educational capabilities. First, more extensive studies in school contexts and workshops, with stable cohorts that allow measuring longitudinal learning rather than only single-session performance. Second, improvements to SCuLPTER as a development tool and reference guide: a more informative error-handling system, supporting tooling such as a language server with syntax highlighting and command completion, and a visualiser that previews the final assembly before printing the pieces. Third, tools aimed specifically at artists: exporting a sculpture's blocks as 3D models ready for printing, and a *sculpture scanner* able to read a physical construction and reconstruct the corresponding program via computer vision. Fourth, the exploration of SCuLPT in expanded artistic contexts such as interactive installations, performances and multidisciplinary workshops.

This last line, the language as an object of *programming experience*, where what matters is not only what is computed but what it feels like to do so, is the most promising. The physicality and geometric abstraction of SCuLPT's statements are, more than a quirk, an invitation to rethink what it means to create with code, who can do it and why they would. Code as art, art as code.

Bibliography

The references are sorted alphabetically by first author.

[1] Samuel Aaron, Alan F. Blackwell, and Pamela Burnard. The development of Sonic Pi and its use in educational partnerships: Co-creating pedagogies for learning computer programming. *Journal of Music, Technology and Education*, 9(1):75–94, 2016. ISSN 1752-7074. DOI 10.1386/jmte.9.1.75_1.

[2] Andrew Begel. LogoBlocks: A graphical programming language for interacting with the world. In *Electrical Engineering and Computer Science Department, MIT*, Cambridge, MA, 1996.

[3] Yorah Bosse and Marco Aurélio Gerosa. Why is programming so difficult to learn? Patterns of difficulties related to programming learning mid-stage. *ACM SIGSOFT Software Engineering Notes*, 41(6):1–6, 2017. ISSN 0163-5948. DOI 10.1145/3011286.3011301.

[4] Nick Collins, Alex McLean, Julian Rohrhuber, and Adrian Ward. Live coding in laptop performance. *Organised Sound*, 8(3):321–330, 2003. DOI 10.1017/S135577180300030X.

[5] Lyn Dupré. *BUGS in Writing: A Guide to Debugging Your Prose*. Revised edition, 1998. ISBN 0-201-37921-X.

[6] Neil Fraser. Ten things we've learned from Blockly. In *Proceedings of the IEEE Blocks and Beyond Workshop*, pages 49–50, 2015. DOI 10.1109/BLOCKS.2015.7369000.

[7] Brian Harvey, Daniel D. Garcia, Tiffany Barnes, Nathaniel Titterton, Daniel Armendariz, Luke Segars, Eugene Lemon, Sean Morris, and Josh Paley. Snap! (Build Your Own Blocks).

In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE)*, page 759, 2013. DOI 10.1145/2445196.2445507.

[8] Michael S. Horn and Robert J. K. Jacob. Designing tangible programming languages for classroom use. In *Proceedings of the 1st International Conference on Tangible and Embedded Interaction (TEI)*, pages 159–162, 2007. DOI 10.1145/1226969.1227003.

[9] Hiroshi Ishii and Brygg Ullmer. Tangible bits: towards seamless interfaces between people, bits and atoms. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, pages 234–241, 1997. DOI 10.1145/258549.258715.

[10] Amy J. Ko. How my broken elbow made the ableism of computer programming personal. *Nature*, September 2023. ISSN 0028-0836. DOI 10.1038/d41586-023-02885-y. URL `https://doi.org/10.1038/d41586-023-02885-y`.

[11] Paul Marshall. Do tangible interfaces enhance learning? *Proceedings of the 1st International Conference on Tangible and Embedded Interaction (TEI)*, pages 163–170, 2007. DOI 10.1145/1226969.1227004.

[12] Maria Montessori. *The Absorbent Mind*. Theosophical Publishing House, Madras, 1949. Reprinted 2012.

[13] Alannah Oleson, Meron Solomon, Christopher Perdriau, and Amy J. Ko. Teaching inclusive design skills with the CIDER assumption elicitation technique. *ACM Transactions on Computer-Human Interaction*, 30(1):1–49, 2023. ISSN 1073-0516. DOI 10.1145/3549074.

[14] Primo Toys. Cubetto: hands-on coding for children. `https://www.primotoys.com/cubetto/`, 2024. Accessed 2026-05-12.

[15] Casey Reas and Ben Fry. *Processing: A Programming Handbook for Visual Designers and Artists*. MIT Press, Cambridge, MA, 2007.

[16] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: programming for all. *Communications of the ACM*, 52(11):60–67, 2009. DOI 10.1145/1592761.1592779.

[17] Anthony Robins, Janet Rountree, and Nathan Rountree. Learning and teaching programming: a review and discussion. *Computer Science Education*, 13(2):137–172, 2003. DOI 10.1076/csed.13.2.137.14200.

[18] William Strunk and E.B. White. *The Elements of Style*. Longman, fourth edition, 2000. ISBN 0-205-30902-X.

[19] Amanda Sullivan and Marina Umaschi Bers. Robotics in the early childhood classroom: experiences with the KIBO robotics kit in preschool classrooms. *International Journal of Technology and Design Education*, 26(1):3–20, 2016. DOI 10.1007/s10798-015-9304-5.

[20] Hideyuki Suzuki and Hiroshi Kato. Interaction-level support for collaborative learning: AlgoBlock—an open programming language. In *Proceedings of the First International Conference on Computer Support for Collaborative Learning (CSCL)*, pages 349–355, 1995. DOI 10.3115/222020.222828.

[21] Tangible Play. Osmo Coding: physical-digital coding games for children. `https://www.playosmo.com/en/coding/`, 2024. Accessed 2026-05-12.

[22] Hongji Yang and Lu Zhang. Promoting creative computing: origin, scope, research and applications. *Digital Communications and Networks*, 2(2):84–91, 2016. ISSN 2352-8648. DOI 10.1016/j.dcan.2016.02.001.

Acronyms